



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Zadanie: Z5

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Projektowanie Prostych Klas

Klasa to podstawowy element programowania obiektowego, który definiuje właściwości (pola) i zachowania (metody) obiektów. Klasa jest jak szablon, na podstawie którego tworzone są obiekty.

Przykład 1:

```
public class Samochod
{
    // Pola klasy
    public string Marka;
    public string Model;
    public int RokProdukcji;

    // Metoda klasy
    public void WyświetlInformacje()
    {
        Console.WriteLine($"Marka: {Marka}, Model: {Model}, Rok Produkcji: {RokProdukcji}");
    }
}
```

Zmienne Obiektowe

Zmienne obiektowe to instancje klas. Każda zmienna obiektowa ma swoje własne wartości pól, niezależnie od innych instancji tej samej klasy.

Przykład 2:

```
Samochod samochod1 = new Samochod();
samochod1.Marka = "Toyota";
```

```
samochod1.Model = "Corolla";  
samochod1.RokProdukcji = 2020;
```

```
Samochod samochod2 = new Samochod();  
samochod2.Marka = "Honda";  
samochod2.Model = "Civic";  
samochod2.RokProdukcji = 2018;
```

```
samochod1.WyswietlInformacje(); // Marka: Toyota, Model: Corolla, Rok Produkcji: 2020  
samochod2.WyswietlInformacje(); // Marka: Honda, Model: Civic, Rok Produkcji: 2018
```

Zakres Widoczności Pól Klasy

Zakres widoczności (modyfikatory dostępu) określają, które elementy klasy są dostępne z zewnątrz. Najczęściej używane modyfikatory to public, private, protected.

Przykład 3:

```
public class Osoba  
{  
    // Pole publiczne - dostępne z zewnątrz  
    public string Imie;  
  
    // Pole prywatne - dostępne tylko wewnątrz klasy  
    private int wiek;  
  
    // Metoda publiczna - dostępna z zewnątrz  
    public void UstawWiek(int nowyWiek)  
    {  
        wiek = nowyWiek;  
    }  
  
    // Metoda publiczna - dostępna z zewnątrz  
    public void WyswietlInformacje()  
    {  
        Console.WriteLine($"Imię: {Imie}, Wiek: {wiek}");  
    }  
}
```

Metody Klasy

Metody to funkcje zdefiniowane wewnątrz klasy, które operują na polach tej klasy. Mogą przyjmować parametry i zwracać wartości.

Przykład 4:

```
public class Kalkulator  
{  
    // Metoda dodająca dwie liczby  
    public int Dodaj(int a, int b)  
    {  
        return a + b;  
    }  
  
    // Metoda odejmująca dwie liczby  
    public int Odejmij(int a, int b)  
    {  
        return a - b;  
    }  
}
```

```
}
```

Dziedziczenie

Dziedziczenie pozwala na tworzenie nowych klas na podstawie istniejących, co umożliwia ponowne wykorzystanie kodu.

Przykład 5:

```
public class Zwierze
{
    public string Nazwa { get; set; }
    public int Wiek { get; set; }

    public void WyświetlInformacje()
    {
        Console.WriteLine($"Nazwa: {Nazwa}, Wiek: {Wiek}");
    }
}

public class Pies : Zwierze
{
    public string Rasa { get; set; }

    public void Szczekaj()
    {
        Console.WriteLine("Woof! Woof!");
    }
}

public class Kot : Zwierze
{
    public string Kolor { get; set; }

    public void Mialcz()
    {
        Console.WriteLine("Meow! Meow!");
    }
}

// Użycie
Pies pies = new Pies { Nazwa = "Burek", Wiek = 3, Rasa = "Labrador" };
pies.WyświetlInformacje();
pies.Szczekaj();

Kot kot = new Kot { Nazwa = "Mruczek", Wiek = 2, Kolor = "Czarny" };
kot.WyświetlInformacje();
kot.Mialcz();
```

Polimorfizm

Polimorfizm pozwala na używanie obiektów różnych klas pochodnych w taki sam sposób, jak obiektów klasy bazowej.

Przykład 6:

```
public class Kształt
{
    public virtual void Rysuj()
    {
```

```

        Console.WriteLine("Rysowanie kształtu");
    }
}

public class Kolo : Kształt
{
    public override void Rysuj()
    {
        Console.WriteLine("Rysowanie koła");
    }
}

public class Kwadrat : Kształt
{
    public override void Rysuj()
    {
        Console.WriteLine("Rysowanie kwadratu");
    }
}

// Użycie
Kształt[] kształty = { new Kolo(), new Kwadrat() };

foreach (Kształt kształt in kształty)
{
    kształt.Rysuj();
}

```

Interfejsy

Interfejsy definiują kontrakty, które klasy muszą implementować. Pozwalają na tworzenie bardziej elastycznych i modularnych aplikacji.

Przykład 7:

```

public interface IZwierze
{
    void Jedz();
    void Spij();
}

public class Pies : IZwierze
{
    public void Jedz()
    {
        Console.WriteLine("Pies je");
    }

    public void Spij()
    {
        Console.WriteLine("Pies śpi");
    }
}

public class Kot : IZwierze
{
    public void Jedz()
    {

```

```

        Console.WriteLine("Kot je");
    }

    public void Spij()
    {
        Console.WriteLine("Kot śpi");
    }
}

```

// Użycie

```

IZwierze pies = new Pies();
pies.Jedz();
pies.Spij();

```

```

IZwierze kot = new Kot();
kot.Jedz();
kot.Spij();

```

ZADANIA

Zadanie 1: Interfejs IPracownik

- o Utwórz interfejs IPracownik z metodami Pracuj i Odpoczywaj.
- o Utwórz klasy Nauczyciel i Lekarz, które implementują interfejs IPracownik.
- o Utwórz obiekty tych klas i wywołaj metody Pracuj i Odpoczywaj.

Zadanie 2: Polimorfizm z Klasą Pojazd

- o Utwórz klasę Pojazd z metodą Jazda.
- o Utwórz klasy Samochod i Rower, które dziedziczą po klasie Pojazd i nadpisują metodę Jazda.
- o Utwórz tablicę obiektów klasy Pojazd i wywołaj metodę Jazda dla każdego z nich.

Zadanie 3: Klasa Pracownik i Dziedziczenie

- o Utwórz klasę Pracownik z polami Imie, Nazwisko, Stanowisko.
- o Utwórz klasy Programista i Kierownik, które dziedziczą po klasie Pracownik.
- o Dodaj specyficzne metody dla każdej z klas pochodnych.
- o Utwórz obiekty tych klas i wyświetl ich informacje.

Zadanie 4: Klasa BankAccount

- o Utwórz klasę BankAccount z polami NumerKonta, Saldo.
- o Dodaj metody Wplata i Wyplata, które będą odpowiednio zwiększać i zmniejszać saldo.
- o Utwórz obiekt klasy BankAccount, wykonaj kilka operacji wpłaty i wypłaty, a następnie wyświetl saldo.

Zadanie 5: Klasa Prostokat

- o Utwórz klasę Prostokat z polami Dlugosc i Szerokosc.
- o Dodaj metody ObliczPole i ObliczObwod, które zwrócą odpowiednio pole i obwód prostokąta.
- o Utwórz obiekt klasy Prostokat, ustaw wartości pól i wyświetl pole oraz obwód.

Zadanie 6: Klasa Student

- o Utwórz klasę Student z polami Imie, Nazwisko, NumerIndeksu.
- o Dodaj metodę WyświetlInformacje, która wyświetli informacje o studencie.
- o Utwórz kilka obiektów klasy Student i wyświetl ich informacje.

Zadanie 7: Klasa Samochod

- o Utwórz klasę Samochod z polami Marka, Model, RokProdukcji.
- o Dodaj metodę WyświetlInformacje, która wyświetli informacje o samochodzie.
- o Utwórz kilka obiektów klasy Samochod i wyświetl ich informacje.